

WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development

STUDENT & COLLEGE MANAGEMENT SYSTEM

A relational database application for admissions, academics, attendance, examinations, grades and fee records

Item	Details
Program	B.Tech CSE (Artificial Intelligence & Machine Learning)
Section	AIML Rhinos
Semester	III
Academic Year	2026–27
Submitted by	Vardan Desai
Register Number	25WU0104029
Guidance	Course Faculty
Date of Submission	October 2026

Repository: <https://github.com/404Vardan/dbms-pbl>

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course. The project gave me a practical setting in which the concepts studied in class could be connected to an actual relational database application.

I am thankful to the course faculty for the guidance and feedback that helped shape the project from the problem definition and ER modelling stage into a working web prototype. The work required me to think about data relationships, normalization, integrity constraints, SQL operations, transactions, reporting and application-level database access as parts of one system rather than as separate exercises.

I also acknowledge the official MySQL, Python Connector/Python, React and PyJWT documentation used while preparing the implementation and database connectivity sections. These references were consulted to keep the technical description consistent with the actual software used in the project.

UNDERTAKING

I hereby declare that the project entitled “Student & College Management System”, submitted as part of the Project-Based Learning activity for the Database Management Systems course, is my individual academic project carried out under the guidance of the course faculty.

I undertake that the database design, SQL scripts, application prototype, test evidence and report have been prepared for academic purposes. Wherever external references or technical documentation have been consulted, appropriate acknowledgement has been provided. The implementation described in this report corresponds to the version of the project maintained in the public GitHub repository at the time of submission.

The report does not claim production deployment or institutional-scale validation. The reported results focus on the database behaviour and application workflows implemented for the PBL prototype.

Declaration detail	Value
Name	Vardan Desai
Register Number	25WU0104029
Program	B.Tech CSE (AIML)
Section	AIML Rhinos
Place	Hyderabad
Date	October 2026

TABLE OF CONTENTS

Contents	Page
Abstract	8
List of Tables	5
List of Figures	6
List of Acronyms	7
1. Problem Statement, Solution Strategy and ER Diagram	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	11
1.6 Entity-Relationship Diagram (ERD)	12
1.7 Entity and Relationship Summary	13
2. Logical Database Design	14
2.1 Relational Schema	14
2.2 Keys and Integrity Constraints	15
2.3 Functional Dependencies	15
2.4 Normalization up to 3NF	15
2.5 Data Dictionary	16
2.6 Implementation of Logical Design: DDL and Sample Data	17
3. Application Development Using Logical Database Design	18
3.1 Application Overview	18
3.2 Development Environment and Database Connectivity	18
3.3 Application Modules and Database Mapping	19
3.4 CRUD Operations	19
3.5 Business Rules, Validation and Error Handling	20
3.6 Reports and Database Operations	20
3.7 Application Screens	21
3.8 Testing and Results	21
4. Conclusion	22
4.1 Limitations and Future Enhancements	22
5. References	23
6. Appendices	24

LIST OF TABLES

No.	Title	Page
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	12
Table 4	Entity and Relationship Summary	13
Table 5	Relational Schema	14
Table 6	Keys and Integrity Constraints	15
Table 7	Normalization Status up to 3NF	15
Table 8	Data Dictionary	16
Table 9	Database Implementation Environment	17
Table 10	Application Development Environment	18
Table 11	Application Module-to-Database Mapping	19
Table 12	CRUD Operations	19
Table 13	Business Rules, Validation and Error Handling	20
Table 14	Reports and Database Operations	20
Table 15	Test Cases	21

LIST OF FIGURES

No.	Title	Page
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	Application-to-Database Architecture	11
Figure 3	Transaction Validation Flow	20
Figure 4	Reporting Data Flow	20

LIST OF ACRONYMS

Acronym	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FK	Foreign Key
JWT	JSON Web Token
PK	Primary Key
RDBMS	Relational Database Management System
SQL	Structured Query Language
3NF	Third Normal Form
UI	User Interface

ABSTRACT

The Student & College Management System (SCMS) is a database application developed as a Database Management Systems Project-Based Learning project to manage common college operations through one relational data model. The system covers student admission, guardian records, departments and programmes, courses, faculty, semesters, sections, course registration, attendance, examinations, grades, fee bills and payments. The conceptual design is implemented as 15 related MySQL tables, supported by primary and foreign keys, UNIQUE, NOT NULL and CHECK constraints, database triggers, stored procedures and reporting views. A React and Vite frontend communicates with a Python and Flask backend, which uses MySQL Connector/Python for database access. The application includes a login flow using bcrypt password verification and JWT-based session data. Business rules such as section capacity, duplicate registration, attendance-date validation, automatic grade calculation and fee overpayment are implemented at the database layer. The completed prototype demonstrates how normalization and database constraints can be translated into day-to-day application workflows while keeping academic and financial reports derived from the same underlying data.

Keywords: Student Information System, MySQL, Relational Database, ER Diagram, Normalization, SQL

1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

College information is usually spread across several operational areas. Admissions maintain student and guardian details, academic teams maintain courses and sections, faculty members record attendance and marks, and accounts staff maintain fee records. When these records are kept independently, the same facts can be entered more than once and questions that combine multiple areas require manual cross-checking. A database-based solution provides a common structure in which a student, course, section or fee bill can be referenced consistently.

The SCMS project addresses this problem by using a single relational schema for the major academic and financial entities. Relationships are represented explicitly through foreign keys, while constraints and triggers prevent several invalid states. MySQL supports primary and foreign keys, CHECK constraints and stored database objects such as triggers, procedures and views [1], [2].

1.2 Objectives

- Design a normalized relational database for core college operations and maintain the main relations at Third Normal Form (3NF).
- Implement primary keys, foreign keys, UNIQUE, NOT NULL and CHECK constraints for data integrity.
- Develop a web application that performs student admission, course registration, attendance, examinations, grading and fee transactions.
- Use stored procedures and triggers for multi-step and row-dependent business rules.
- Generate cross-module academic and financial reports from SQL views and joins.
- Provide a simple login flow for the prototype and protect the application routes after authentication.

1.3 Scope and Assumptions

Area	In scope	Assumption / boundary
Admissions	Student + guardian + programme records	Sample academic data; local MySQL database
Academics	Departments, programmes, courses, faculty, semesters, sections	No timetable conflict engine
Transactions	Registration, attendance, examinations, grades	One academic session can be represented by semester rows
Finance	Fee bills, payments and due calculations	Payment methods are controlled by CHECK constraint
Reporting	Occupancy, attendance, results, history,	Reports are derived from base data and

Area	In scope	Assumption / boundary
	dues and department summary	views
Authentication	Login with role stored in user_account	Prototype-level authorization; deeper API authorization is future scope
Out of scope	Hostel, library, payroll, alumni, online proctoring	Not required for this PBL prototype

Table 1. Scope and Assumptions

1.4 Proposed Solution Strategy

The project follows a database-first implementation sequence. The conceptual entities are mapped to relational tables; primary and foreign keys define the relationships; domain constraints restrict invalid values; and triggers are added for rules that depend on other rows, such as registration capacity or cumulative payments. Stored procedures are used when an operation spans multiple related changes, such as adding a student with guardian information. Finally, SQL views provide reusable reporting queries that are consumed by the Flask routes.

Layer	Implementation	Purpose
Presentation	React + Vite	Forms, tables, navigation and report screens
Application	Python + Flask	REST endpoints and input handling
Database	MySQL 8.0+ / InnoDB	Persistent relational data and integrity rules
Database objects	Triggers, procedures, views	Business rules, atomic operations and reports
Authentication	bcrypt + PyJWT	Password verification and signed login token

Table 2. Proposed Solution Strategy

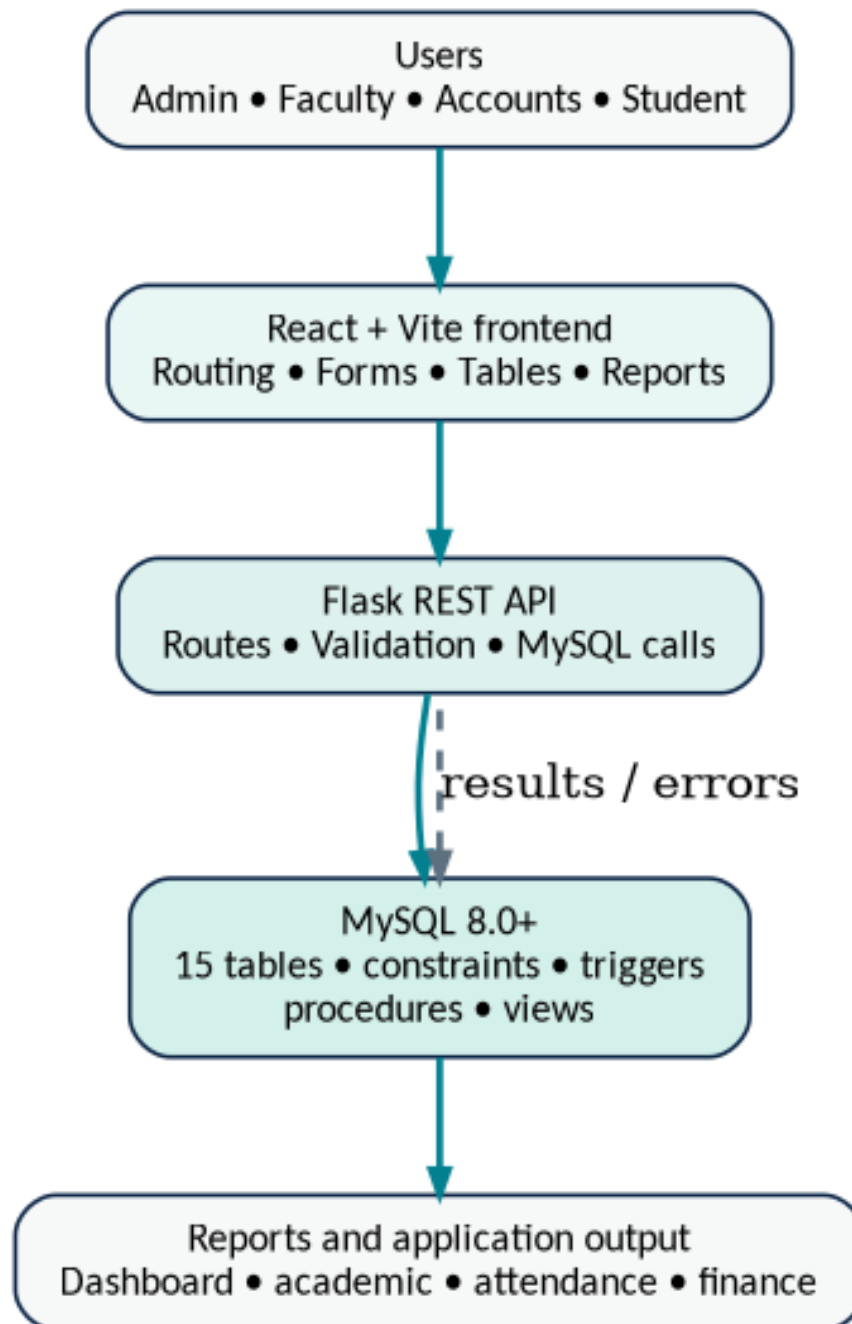


Figure 2. Application-to-Database Architecture

1.5 Functional Requirements

ID	Requirement	Implementation evidence
FR1	Student admission	Create student and guardian records using a database procedure.
FR2	Academic setup	Maintain departments, programmes, courses, faculty, semesters and sections.

ID	Requirement	Implementation evidence
FR3	Registration	Register active students into sections with capacity and duplicate checks.
FR4	Attendance	Record daily attendance and reject invalid dates or dropped registrations.
FR5	Examinations	Record marks and derive the letter grade through a MySQL trigger.
FR6	Fees	Create fee bills and record payments without allowing overpayment.
FR7	Reports	Display occupancy, attendance, result, history, dues and department reports.
FR8	Authentication	Login using pre-seeded user accounts and protect application routes.

Table 3. Functional Requirements

1.6 Entity-Relationship Diagram (ERD)

Figure 1 shows the final relational entity structure implemented in MySQL. Fourteen operational domain tables model the academic and finance workflow, while user_account is an additional application table for login identity and role information. The central many-to-many relationship between students and sections is resolved through registration. Attendance and examination records depend on a registration, while grades depend on examinations. Fee bills connect a student to a semester and payments connect to an individual bill.

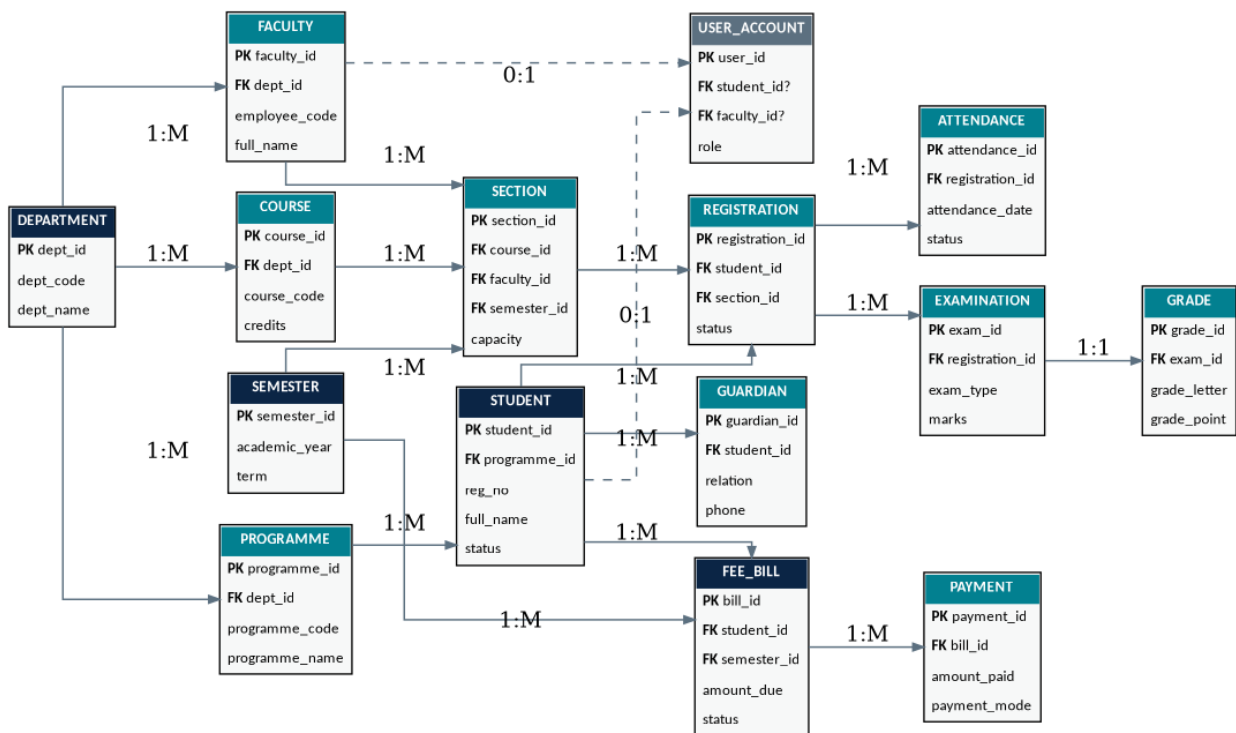


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Entity	Purpose	Cardinality	Main relationship
department	Academic department	1:M	programme, faculty, course
programme	Degree programme	1:M	student
faculty	Teaching staff	1:M	section
course	Course catalogue	1:M	section
semester	Academic term	1:M	section, fee_bill
section	Offering of a course	1:M	registration
student	Student master record	1:M	guardian, registration, fee_bill
guardian	Guardian contact	M:1	student
registration	Student-section enrollment	1:M	attendance, examination
attendance	Class attendance	M:1	registration
examination	Assessment record	1:1	grade
grade	Derived grade	1:1	examination
fee_bill	Semester fee bill	1:M	payment
payment	Payment transaction	M:1	fee_bill
user_account	Login identity	0:1	student/faculty

Table 4. Entity and Relationship Summary

2. LOGICAL DATABASE DESIGN

2.1 Relational Schema

The final logical schema contains 15 relations. Surrogate integer keys are used as the primary identifiers, while business identifiers such as reg_no, course_code and employee_code are protected by UNIQUE constraints. Foreign keys follow the parent-child direction of the ER model. Registration, attendance, examination and payment are transactional tables because their rows record events or actions against master records.

Relation	Primary Key	Foreign Keys	Role in system
department	dept_id	—	department master
programme	programme_id	dept_id	degree programme
faculty	faculty_id	dept_id	faculty master
course	course_id	dept_id	course master
semester	semester_id	—	academic term
section	section_id	course_id, faculty_id, semester_id	course offering
student	student_id	programme_id	student master
guardian	guardian_id	student_id	guardian details
registration	registration_id	student_id, section_id	course enrollment
attendance	attendance_id	registration_id	attendance event
examination	exam_id	registration_id	exam marks
grade	grade_id	exam_id	derived grade
fee_bill	bill_id	student_id, semester_id	fee obligation
payment	payment_id	bill_id	payment transaction
user_account	user_id	student_id?, faculty_id?	login identity

Table 5. Relational Schema

2.2 Keys and Integrity Constraints

The schema uses a combination of entity keys and domain constraints. MySQL permits foreign keys and CHECK constraints in CREATE TABLE definitions, while InnoDB is the default transactional storage engine used in this project [1]. The following constraints are important because they preserve the relationships even when a request reaches the database directly.

Type	Examples in SCMS	Purpose
PK	Every table has a single surrogate primary key.	Unique row identity.
FK	Child tables reference required parent rows.	Referential integrity.
UNIQUE	reg_no, email, course_code and selected compound keys are unique.	Prevents duplicate business identifiers.
NOT NULL	Required academic and transactional attributes are mandatory.	Avoids incomplete records.
CHECK	Domain values for status, term, marks, payment mode and amounts are restricted.	Blocks invalid domain values.
DEFAULT	Default status, dates and timestamps are used where appropriate.	Reduces repetitive application logic.

Table 6. Keys and Integrity Constraints

2.3 Functional Dependencies

Functional dependencies express which attributes are determined by a key. In SCMS, each surrogate primary key determines the non-key attributes of its relation. A few representative dependencies are listed below; the same pattern applies to the other relations.

Determinant	Dependent attributes
dept_id	dept_code, dept_name, office_email
programme_id	dept_id, programme_code, programme_name, duration_years, status
course_id	dept_id, course_code, course_name, credits, course_type, status
student_id	programme_id, reg_no, full_name, dob, email, phone, admission_date, status
registration_id	student_id, section_id, registered_on, status
exam_id	registration_id, exam_type, exam_date, max_marks, marks
bill_id	student_id, semester_id, bill_date, amount_due, due_date, status
payment_id	bill_id, payment_date, amount_paid, payment_mode, reference_no
user_id	email, password_hash, role, full_name, student_id, faculty_id, created_at

2.4 Normalization up to Third Normal Form (3NF)

Normalization was used to keep facts in the relations where they belong. The schema avoids repeating groups, separates many-to-many relationships through registration, and avoids storing derived or repeated details such as the course name inside attendance or examination rows. The final design is treated as 3NF for the academic scope of the project because non-key attributes depend on the key of their own relation and the model does not intentionally keep transitive descriptive facts in child transactions.

Stage	Design decision	Result
1NF	Each attribute is atomic; no repeating group is stored in a single column.	Rows can be addressed consistently.
2NF	Transactions are separated from master entities; registration resolves student-section association.	Non-key attributes describe the whole row.
3NF	Programme, department, course, faculty and fee data are stored in separate relations.	Fewer update and insertion anomalies.

Table 7. Normalization Status up to 3NF

2.5 Data Dictionary

Table	Column	Type	Constraints	Meaning
department	dept_id	INT	PK	Department master

Table	Column	Type	Constraints	Meaning
department	dept_code	VARCHAR(10)	UNIQUE, NOT NULL	Short code
department	dept_name	VARCHAR(100)	UNIQUE, NOT NULL	Department name
programme	programme_id	INT	PK	Programme key
programme	dept_id	INT	FK, NOT NULL	Parent department
programme	programme_code	VARCHAR(15)	UNIQUE	Programme code
faculty	faculty_id	INT	PK	Faculty key
faculty	dept_id	INT	FK	Faculty department
faculty	employee_code	VARCHAR(15)	UNIQUE	Employee code
course	course_id	INT	PK	Course key
course	dept_id	INT	FK	Course department
course	course_code	VARCHAR(12)	UNIQUE	Course code
course	credits	TINYINT	CHECK > 0	Credit value

Table	Column	Type	Constraints	Meaning
semester	semester_id	INT	PK	Semester key
semester	academic_year	VARCHAR(9)	NOT NULL	Academic year
semester	term	VARCHAR(10)	CHECK	Odd / Even / Summer
section	section_id	INT	PK	Section key
section	course_id	INT	FK	Course offered
section	faculty_id	INT	FK	Assigned faculty
section	semester_id	INT	FK	Offering term
section	capacity	SMALLINT	CHECK > 0	Maximum students
student	student_id	INT	PK	Student key
student	programme_id	INT	FK	Programme
student	reg_no	VARCHAR(20)	UNIQUE	Registration number
student	status	VARCHAR(12)	CHECK	Active / Inactive / Graduated / Withdrawn
guardian	guardian_id	INT	PK	Guardian key

Table	Column	Type	Constraints	Meaning
guardian	student_id	INT	FK	Related student
registration	registration_id	INT	PK	Registration key
registration	student_id	INT	FK	Student
registration	section_id	INT	FK	Section
attendance	attendance_id	INT	PK	Attendance key
attendance	registration_id	INT	FK	Registration
attendance	status	VARCHAR(7)	CHECK	Present / Absent / Late
examination	exam_id	INT	PK	Exam key
examination	registration_id	INT	FK	Registration
examination	marks	DECIMAL(5,2)	CHECK	Score 0..max_marks
grade	grade_id	INT	PK	Grade key
grade	exam_id	INT	FK, UNIQUE	One grade per exam
fee_bill	bill_id	INT	PK	Fee bill key
fee_bill	student_id	INT	FK	Billed student
fee_bill	amount_due	DECIMAL(10,2)	CHECK >= 0	Amount due
payment	payment_id	INT	PK	Payment key
payment	bill_id	INT	FK	Bill paid against
payment	amount_paid	DECIMAL(10,2)	CHECK > 0	Payment amount
user_account	user_id	INT	PK	Login user
user_account	role	VARCHAR(15)	CHECK	admin / faculty / student / accounts

Table 8. Data Dictionary

2.6 Implementation of Logical Design: DDL and Sample Data

The database implementation is divided into four SQL layers so that the build can be reproduced in a clear order. The schema script creates the database and tables. The trigger script adds business rules. The procedure script contains multi-step transactions. The view script adds

reusable report queries. Finally, seed.sql inserts sample data for demonstration and testing. This separation keeps the core DDL readable and makes the individual DBMS features easier to inspect.

File	Purpose	Main objects
schema.sql	Create database and relations	15 tables, keys, constraints
triggers.sql	Database business rules	registration, attendance, payment, grade triggers
procedures.sql	Atomic multi-step operations	sp_admit_student, sp_record_exam_result
views.sql	Reusable reporting queries	8 views
seed.sql	Demonstration records	linked academic and fee sample data

Table 9. Database Implementation Environment

```
CREATE TABLE registration (
  registration_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  section_id INT NOT NULL,
  registered_on DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status VARCHAR(12) NOT NULL DEFAULT 'Registered',
  CONSTRAINT fk_registration_student
    FOREIGN KEY (student_id) REFERENCES student(student_id),
  CONSTRAINT fk_registration_section
    FOREIGN KEY (section_id) REFERENCES section(section_id),
  CONSTRAINT uq_registration_student_section
    UNIQUE (student_id, section_id)
) ENGINE=InnoDB;
```

The complete SQL package remains in the GitHub repository under database/. The code shown above is representative of the actual DDL and demonstrates the way keys, references and uniqueness rules are defined.

3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The frontend is implemented in React with Vite. It provides the screens through which users search records, submit forms, record transactions and open reports. The backend is a Flask application with route modules separated by functional area. Database access is centralized in server/db.py, which opens a MySQL Connector/Python connection, executes parameterized queries and converts database values into JSON-friendly values. Connector/Python is the Python driver used to communicate with MySQL databases [3].

The application currently contains the following operational screens: Dashboard, Students, Student Details, Departments, Courses, Faculty, Semesters, Sections, Registration, Attendance, Examinations, Fees and Reports. The Login page is the entry point to the protected application routes. React supports component-based user interfaces and routing is used here to separate the login route from the main layout [4].

3.2 Development Environment and Database Connectivity

Component	Technology / version	Use in project
Frontend runtime	React 19 + Vite	Browser application and build tool
Backend	Python 3 + Flask	REST API and request handling
Database	MySQL 8.0+ / InnoDB	Persistent relational storage
Driver	mysql-connector-python	Python-to-MySQL connection
Password handling	bcrypt	Verification of stored password hashes
Session token	PyJWT	JWT creation for login session data
Package management	npm + pip	JavaScript and Python dependencies
Source control	GitHub	Public repository and project source

Table 10. Application Development Environment

The database connection parameters are read from environment variables rather than being hard-coded into the connection helper. The frontend calls Flask endpoints under /api, and the route handlers perform database operations through the shared query helper. The authentication route checks the selected email in user_account, verifies the password hash with bcrypt and returns a signed JWT payload using PyJWT. PyJWT documents JWT encoding and decoding through jwt.encode() and jwt.decode() [5].

3.3 Application Modules and Database Mapping

Module	Main operation	Database objects
Dashboard	Aggregates and recent transactions	student, registration, attendance, fee_bill,

Module	Main operation	Database objects
		payment
Students	Student directory, details, admission	student, guardian, programme, v_student_academic_history, v_fee_dues
Departments	Departments and programmes	department, programme
Courses	Course catalogue	course, department
Faculty	Faculty directory	faculty, department
Semesters	Academic calendar	semester
Sections	Course offerings	section, course, faculty, semester
Registration	Enroll / drop course sections	registration, student, section
Attendance	Daily attendance + summary	attendance, registration, v_attendance_summary
Examinations	Marks and grades	examination, grade, sp_record_exam_result
Fees	Bills and payments	fee_bill, payment, v_fee_dues
Reports	Analytical views	all reporting views

Table 11. Application Module-to-Database Mapping

3.4 CRUD Operations

Operation	Examples	Implementation
Create	Students, courses, sections, registrations, attendance, exams, fee bills, payments	INSERT / procedure calls
Read	Directories, details, dashboard and reports	SELECT / JOIN / views
Update	Student details, registration status, attendance status, exam marks	UPDATE / procedure calls
Delete / deactivate	Registration drop and controlled student deactivation	UPDATE status rather than physical deletion in key workflows

Table 12. CRUD Operations

3.5 Business Rules, Validation and Error Handling

A design choice in the final prototype is to let the database decide rules that depend on related rows. The application validates required input fields first, but the INSERT or procedure call is still subject to MySQL constraints and triggers. When a trigger raises SQLSTATE 45000, the Flask route returns the database error to the frontend instead of silently accepting the transaction.

Rule	Database object	Behaviour
Duplicate course registration	trg_registration_before_insert	If the same student is already registered for the same course in the same semester, the new registration is rejected.
Section capacity	trg_registration_before_insert	If current occupancy reaches capacity, the new registration is rejected.
Active student check	trg_registration_before_insert	Inactive, graduated or withdrawn students cannot be registered.
Attendance date	trg_attendance_before_insert	Future dates and dates outside the semester range are rejected.
Grade derivation	trg_examination_after_insert / update	Marks are converted to the stored grade letter and grade point.
Fee overpayment	trg_payment_before_insert	Payment above the remaining bill balance is rejected.
Fee status	trg_payment_after_insert	Bill status becomes Paid, Partially Paid or Unpaid.

Rule	Database object	Behaviour
Atomic admission	sp_admit_student	Student and guardian creation is committed together or rolled back.

Table 13. Business Rules, Validation and Error Handling

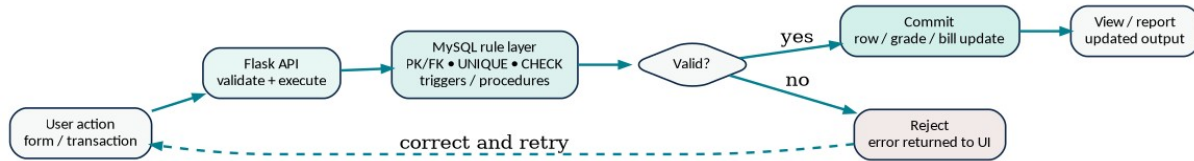


Figure 3. Transaction Validation Flow

3.6 Reports and Database Operations

The Reports page does not store separate report copies. Instead, the Flask report route selects from SQL views defined in database/views.sql. The approach keeps the report output tied to the same normalized data used by the transactional screens. This also makes the purpose of SQL JOIN, GROUP BY and aggregate functions visible in the application.

Report	SQL view	Main operation
Section occupancy	v_section_occupancy	COUNT / capacity / occupancy percentage
Attendance summary	v_attendance_summary	attendance percentage and shortage flag
Exam details	v_exam_details	marks, percentage and grade
Result analysis	v_result_analysis	average, highest, lowest and pass percentage
Academic history	v_student_academic_history	student-wise course, marks, grade and attendance
Fee dues	v_fee_dues	amount due, paid, balance and overdue status
Student dues	v_student_dues	student-level fee summary
Department summary	v_department_summary	programme, faculty, course and student totals

Table 14. Reports and Database Operations



Figure 4. Reporting Data Flow

3.7 Application Screens

The completed prototype contains a consistent sidebar layout and separate screens for the main database modules. The screen descriptions below correspond to the current page structure in src/pages/. The figures are described by function rather than presented as decorative mockups; the live interface is available from the project repository and can be demonstrated locally with the setup steps given in this report.

Screen	Representative task	Database evidence
Login	Select demo account and authenticate	user_account + JWT
Dashboard	View current system counts and recent finance activity	aggregate queries
Students	Search, view and admit students	student + guardian + programme
Registration	Select active student and section	registration + capacity / duplicate triggers
Attendance	Select section and mark daily status	attendance + attendance summary view
Examinations	Enter marks and view grade	examination + grade trigger / procedure
Fees	Create bill, record payment and view balance	fee_bill + payment + fee views
Reports	Open cross-module analytical outputs	SQL views

A useful point for demonstration is that the forms are not the final source of truth. For example, the registration screen may display a section as available, but the database trigger still rechecks capacity and duplicate course enrollment at insert time. The same principle applies to attendance, marks and payment rules.

3.8 Testing and Results

Testing for the final prototype focuses on the business rules most likely to produce an incorrect record if the application relied only on frontend validation. The current source code and seeded data provide clear paths for each of the following cases. Statuses below describe the implemented behaviour that the reviewer should reproduce during the live demonstration; no performance benchmark is claimed.

ID	Test	Input / condition	Expected result	Status
TC1	Valid registration	Active student + available section	Registration row created	Implemented
TC2	Duplicate course registration	Same student + same course + same semester	Trigger raises an error	Implemented
TC3	Full section	Section at capacity	Trigger raises an error	Implemented
TC4	Invalid attendance	Future date / outside semester / dropped registration	Trigger raises an error	Implemented
TC5	Automatic grade	Marks entered through examination workflow	Grade row created / updated	Implemented
TC6	Fee overpayment	Payment greater than remaining balance	Trigger raises an error	Implemented
TC7	Fee status	Valid payment inserted	Bill status recalculated	Implemented
TC8	Student admission	Valid student + guardian input	Procedure creates both records atomically	Implemented

Table 15. Test Cases

The seeded data is useful for demonstration because it contains examples of a full section, an attendance-shortage case, examination marks that map to different grades, and fee bills with paid and unpaid balances. The live demonstration should use one positive transaction followed by one negative case for each major database rule so that the database response is visible.

4. CONCLUSION

The Student & College Management System brings the main academic and financial records used in the project into one relational model and connects that model to a working web application. The important DBMS ideas are visible in the actual implementation: entity relationships are reflected in foreign keys, business identifiers are protected with UNIQUE constraints, domain values are restricted with CHECK constraints, and cross-row rules are handled using triggers.

The project also shows the practical relationship between database design and application development. The frontend provides the forms and reports, Flask provides the API layer, and MySQL remains responsible for persistent data and the final database rules. Stored procedures cover multi-step operations while views keep reporting queries reusable. The result is a compact academic system that can be demonstrated end-to-end instead of remaining only a schema exercise.

4.1 Limitations and Future Enhancements

Current limitation	Reasonable future enhancement
Prototype-level authorization	Add server-side role middleware to enforce permissions on every API route
Local MySQL dependency	Deploy the backend and database using managed hosting
Basic dashboard	Add more detailed trend and cohort reports
Manual data entry	Support CSV import and export with validation
No notification service	Add email or SMS alerts for attendance and fee reminders
No audit history for every change	Add an audit table and transaction logging

The project deliberately does not claim to be a full college ERP. Production use would require stronger authentication controls, server-side authorization, backup policies, audit logging, monitoring and broader testing with real institutional requirements.

5. REFERENCES

- [1] Oracle Corporation, “MySQL 8.0 Reference Manual: CREATE TABLE Statement,” MySQL Documentation, 2026. Available: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>
- [2] Oracle Corporation, “MySQL 8.0 Reference Manual: Stored Objects,” MySQL Documentation, 2026. Available: <https://dev.mysql.com/doc/refman/8.0/en/stored-objects.html>
- [3] Oracle Corporation, “MySQL Connector/Python Developer Guide,” MySQL Documentation, 2026. Available: <https://dev.mysql.com/doc/connector-python/en/>
- [4] Meta Platforms, Inc., “React Documentation,” React, 2026. Available: <https://react.dev/>
- [5] PyJWT Project, “PyJWT Documentation,” 2026. Available: <https://pyjwt.readthedocs.io/en/latest/>
- [6] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.
- [7] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2020.
- [8] 404Vardan, “Student & College Management System — DBMS PBL,” GitHub repository, 2026. Available: <https://github.com/404Vardan/dbms-pbl>

6. APPENDICES

Appendix A. Database Script Package

The repository stores the database implementation as separate SQL files so that the evaluator can inspect and run each layer independently. The execution order is schema.sql, triggers.sql, procedures.sql, views.sql and seed.sql.

File	Purpose	Key contents
database/schema.sql	DDL	database creation, 15 tables, PK/FK, UNIQUE, NOT NULL, CHECK
database/triggers.sql	Integrity rules	registration, attendance, payment, grading
database/procedures.sql	Transactions	student admission and exam result procedure
database/views.sql	Reports	8 SQL views
database/seed.sql	Sample records	departments through payments + demo accounts

A1. Representative DDL

```
CREATE TABLE fee_bill (  
    bill_id INT AUTO_INCREMENT PRIMARY KEY,  
    student_id INT NOT NULL,  
    semester_id INT NOT NULL,  
    bill_date DATE NOT NULL DEFAULT (CURRENT_DATE),  
    amount_due DECIMAL(10,2) NOT NULL,  
    due_date DATE NOT NULL,  
    status VARCHAR(15) NOT NULL DEFAULT 'Unpaid',  
    CONSTRAINT fk_fee_bill_student FOREIGN KEY (student_id) REFERENCES  
student(student_id),  
    CONSTRAINT fk_fee_bill_semester FOREIGN KEY (semester_id) REFERENCES  
semester(semester_id),  
    CONSTRAINT uq_fee_bill_student_semester UNIQUE (student_id, semester_id),  
    CONSTRAINT ck_fee_bill_amount CHECK (amount_due >= 0)  
) ENGINE=InnoDB;
```

A2. Representative Trigger

```
CREATE TRIGGER trg_payment_before_insert  
BEFORE INSERT ON payment  
FOR EACH ROW  
BEGIN  
    DECLARE v_amount_due DECIMAL(10,2);  
    DECLARE v_total_paid DECIMAL(10,2);  
    DECLARE v_remaining DECIMAL(10,2);  
  
    SELECT amount_due INTO v_amount_due FROM fee_bill WHERE bill_id =  
NEW.bill_id;  
    SELECT COALESCE(SUM(amount_paid), 0) INTO v_total_paid FROM payment WHERE  
bill_id = NEW.bill_id;  
    SET v_remaining = v_amount_due - v_total_paid;
```

```

        IF NEW.amount_paid > v_remaining THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Payment Rejected: Total payment cannot exceed the
remaining balance due.';
        END IF;
    END;

```

A3. Representative Procedure

```

CALL sp_admit_student(
    programme_id, reg_no, full_name, dob, email, phone, admission_date,
    guardian_name, guardian_relation, guardian_phone, guardian_email,
    guardian_address,
    @student_id
);

CALL sp_record_exam_result(
    registration_id, exam_type, exam_date, marks, max_marks, @exam_id
);

```

A4. Representative Reporting View

```

CREATE VIEW v_student_dues AS
SELECT student_id, reg_no, student_name, student_status, programme_code,
       COUNT(*) AS bills, SUM(amount_due) AS total_billed,
       SUM(amount_paid) AS total_paid, SUM(balance) AS outstanding
FROM v_fee_dues
GROUP BY student_id, reg_no, student_name, student_status, programme_code;

```

The repository contains the complete versions of these scripts. Appendix A intentionally shows representative sections so the report remains readable; the GitHub source is the authoritative copy of the full SQL package.

Appendix B. Demo Accounts

Role	Email	Password
Admin	registrar@scms.edu.in	Demo@12345
Faculty	priya.raghavan@scms.edu.in	Demo@12345
Accounts	accounts@scms.edu.in	Demo@12345
Student	meera.nair@students.scms.edu.in	Demo@12345

These accounts are demo credentials intentionally included in seed.sql for academic evaluation. They are not production credentials.

Appendix C. Repository and Run Sequence

```

git clone https://github.com/404Vardan/dbms-pbl.git
cd dbms-pbl

# Create MySQL database and objects

```

```
mysql -u root -p < database/schema.sql
mysql -u root -p < database/triggers.sql
mysql -u root -p < database/procedures.sql
mysql -u root -p < database/views.sql
mysql -u root -p < database/seed.sql
```

```
# Python backend
pip install -r server/requirements.txt
```

```
# React frontend
npm install
npm start
```

Appendix D. Submission Evidence Checklist

- Public GitHub repository accessible at <https://github.com/404Vardan/dbms-pbl>
- README documents the actual React + Flask + MySQL implementation.
- Authentication routes unauthenticated users to the login page and preserves logged-in sessions.
- Database scripts are separated into schema, triggers, procedures, views and seed data.
- Final demonstrations should include at least one successful transaction and one rejected transaction for database rule enforcement.
- Report, three review/final presentations and live prototype should describe the same MySQL implementation.